Intro to OS

OS - The interface between hardware and user

Inside OS

BIOS is not usually scanned by anti virus

Resource Allocation via the OS

GUI - graphic user Interface

CLI - Comand line interface

Module - 2

Linux Operating System

To Learn :

- Architecture of linux
- Different Distribution of linux
- The Shell

Intro to Linux

Open source .

Architecture :

components of Linux

- User
- Application
- Shell
- Filesystem Hierchy Standard
- Kernel
- Hardware

User : the person that interact with the a computer

Application : A Program that perform a specific task

Shell: The Comand line interpreter

Kernal: the Component of the Linux os that manges processes and memory

Hardware: physical components

Linux Distribution

different version of linux

Parent distribution :-

- Red hat Enterprise linux (Cent OS)
- Slackware (SUSE)
- Debian (Kali & linux)

Kali linux : trade mark of offensive security system and is debian derived

Pentest tools in linux :-

- Metasploit :- Exploit vulnerability on machine
- Burp Suit :- use for weakness in web application
- John the Ripper :- used to guess the Password

Digital forencics tools :-

- tcpDump :- is a Command line Package analyzer is use to Capture network traffic
- wireshark - is a GUI Based use to analyze live and Capture network traffic
- Autopsy : tool use to analyze phones and harddrive

More linux Distribution

1. Ubuntu :- an open source, user friendly distribution that is widly used in security has both CLI & GUI

2. Parrot :- also open source similar to Kali

Intro to Shell

Shell - is a command line interpreture

Comand :- An instruction telling the computer to do something

Different types of Shell

Bash :- Defult shell in most linux distribution and user friendly used for basic linux commands as well as larger Projects.

- C Shell (csh)
- Korn shell (Ksh)
- Enhanced C Shell (tcsh)
- Z shell (zsh)

Module - 3

Linux Comand in Bash Shell

To learn

- The Comand line
- Navigating and managing the file system
- Authenticating and authorizing user

Bash : The default shell in most linux distro.

$: is the Promp to enter new Comand

Argument (Linux)

:- A specific information needed by a comand

Core Comands for Navigating & reading files

Root Directory :- The highest level directory in Linux

"/" - represent branch

1. PWD :- Print working directory
2. ls :- list all the file in current Directory
3. CD :- Change directory.

Reading file

1. Cat :- Display content of file
2. head :- Display just leging of file Defult 10 lines
3. tail : end of file Defult 10 lines
4. less : returns one page at a time

filtering the file

1. grep : searches a specified file and return all lines in the file containg a specified string

   Ex :- grep OS update.txt

   1. String that we are looking for
   2. file that is being searched

2. (Piping) "|"

   sends the standard out of one command as standard input to another command for further processing

3. find : search for all the files and directory that meet specific criteria

   Criterias for find can be added by (-)

-name and -iname

and -mtime - to find file modified with Certain Period

Creating and Modifing directories

1. MKDIR → Make directory
2. RMDIR → Remove directory
3. Touch → Creates new file
4. RM → Remove Delete a file

Moving

1. MV → Moves a file or Directory to New location
2. CP → Copies a file or directory to New location

Both take 2 arguments file name/Directory Name and location what to do.

3. Nano :- use to edit files.

Permission commands
Previously, you explored file permissions and the commands that you can use to display and change them. In this reading, you'll review these concepts and also focus on an example of how these commands work together when putting the principle of least privilege into practice.

Reading permissions

In Linux, permissions are represented with a 10-character string. Permissions include:

read: for files, this is the ability to read the file contents; for directories, this is the ability to read all contents in the directory including both files and subdirectories

write: for files, this is the ability to make modifications on the file contents; for directories, this is the ability to create new files in the directory

execute: for files, this is the ability to execute the file if it's a program; for directories, this is the ability to enter the directory and access its files

These permissions are given to these types of owners:

user: the owner of the file

group: a larger group that the owner is a part of

other: all other users on the system

Each character in the 10-character string conveys different information about these permissions. The following table describes the purpose of each character:

Character Example Meaning

1st
drwxrwxrwx

file type
d for directory
- for a regular file

2nd
drwxrwxrwx

read permissions for the user
r if the user has read permissions
- if the user lacks read permissions

3rd
drwxrwxrwx

write permissions for the user
w if the user has write permissions
- if the user lacks write permissions

4th
drwxrwxrwx

execute permissions for the user
x if the user has execute permissions
- if the user lacks execute permissions

5th
drwxrwxrwx

read permissions for the group
r if the group has read permissions
- if the group lacks read permissions

6th
drwxrwxrwx

write permissions for the group
w if the group has write permissions
- if the group lacks write permissions

7th
drwxrwxrwx

execute permissions for the group
x if the group has execute permissions
- if the group lacks execute permissions

8th
drwxrwxrwx

read permissions for other
r if the other owner type has read permissions
- if the other owner type lacks read permissions

9th
drwxrwxrwx

write permissions for other
w if the other owner type has write permissions
- if the other owner type lacks write permissions

10th
drwxrwxrwx

execute permissions for other

x if the other owner type has execute permissions
- if the other owner type lacks execute permissions
Exploring existing permissions

You can use the ls command to investigate who has permissions on files and directories. Previously, you learned that ls displays the names of files in directories in the current working directory.

There are additional options you can add to the ls command to make your command more specific. Some of these options provide details about permissions. Here are a few important ls options for security analysts:

ls -a: Displays hidden files. Hidden files start with a period (.) at the beginning.

ls -l: Displays permissions to files and directories. Also displays other additional information, including owner name, group, file size, and the time of last modification.

ls -la: Displays permissions to files and directories, including hidden files. This is a combination of the other two options.

Changing permissions

The principle of least privilege is the concept of granting only the minimal access and authorization required to complete a task or function. In other words, users should not have privileges that are beyond what is necessary. Not following the principle of least privilege can create security risks.

The chmod command can help you manage this authorization. The chmod command changes permissions on files and directories.

Using chmod

The chmod command requires two arguments. The first argument indicates how to change permissions, and the second argument indicates the file or directory that you want to change permissions for. For example, the following command would add all permissions to login_sessions.txt:

chmod u+rwx,g+rwx,o+rwx login_sessions.txt

If you wanted to take all the permissions away, you could use

chmod u-rwx,g-rwx,o-rwx login_sessions.txt

Another way to assign these permissions is to use the equals sign (=) in this first argument. Using = with chmod sets, or assigns, the permissions exactly as specified. For example, the following command would set read permissions for login_sessions.txt for user, group, and other:

chmod u=r,g=r,o=r login_sessions.txt

This command overwrites existing permissions. For instance, if the user previously had write permissions, these write permissions are removed after you specify only read permissions with =.

The following table reviews how each character is used within the first argument of chmod:

Character

Description

u

indicates changes will be made to user permissions

g

indicates changes will be made to group permissions

o

indicates changes will be made to other permissions

+

Adding and deleting user

① Root user (Super User): A user with elevated privilages to modify the system.

Problem with logging in as a root user

- Security risk
- very easy to make irreversible mistake

Solution

SUDO: temporarily grants elevated permission to specific user

useradd :- Add user to the system

Userdel :- Delete user from the system

useradd

The `useradd` command adds a user to the system. To add a user with the username of `fgarcia` with `sudo`, enter `sudo useradd fgarcia`. There are additional options you can use with `useradd`:

- `-g`: Sets the user's default group, also called their primary group
- `-G`: Adds the user to additional groups, also called supplemental or secondary groups

To use the `-g` option, the primary group must be specified after `-g`. For example, entering `sudo useradd -g security fgarcia` adds `fgarcia` as a new user and assigns their primary group to be `security`.

To use the `-G` option, the supplemental group must be passed into the command after `-G`. You can add more than one supplemental group at a time with the `-G` option. Entering `sudo useradd -G finance,admin fgarcia` adds `fgarcia` as a new user and adds them to the existing `finance` and `admin` groups.

usermod

The `usermod` command modifies existing user accounts. The same `-g` and `-G` options from the `useradd` command can be used with `usermod` if a user already exists.

To change the primary group of an existing user, you need the `-g` option. For example, entering `sudo usermod -g executive fgarcia` would change `fgarcia`'s primary group to the `executive` group.

To add a supplemental group for an existing user, you need the `-G` option. You also need a `-a` option, which appends the user to an existing group and is only used with the `-G` option. For example, entering `sudo usermod -a -G marketing fgarcia` would add the existing `fgarcia` user to the supplemental `marketing` group.

Note: When changing the supplemental group of an existing user, if you don't include the `-a` option, `-G` will replace any existing supplemental groups with the groups specified after `usermod`. Using `-a` with `-G` ensures that the new groups are added but existing groups are not replaced.

There are other options you can use with `usermod` to specify how you want to modify the user, including:

Note: When changing the supplemental group of an existing user, if you don't include the `-a` option, `-G` will replace any existing supplemental groups with the groups specified after `usermod`. Using `-a` with `-G` ensures that the new groups are added but existing groups are not replaced.

There are other options you can use with `usermod` to specify how you want to modify the user, including:

- `-d`: Changes the user's home directory.
- `-l`: Changes the user's login name.
- `-L`: Locks the account so the user can't log in.

The option always goes after the `usermod` command. For example, to change `fgarcia`'s home directory to `/home/garcia_f`, enter `sudo usermod -d /home/garcia_f fgarcia`. The option `-d` directly follows the command `usermod` before the other two needed arguments.

userdel

The `userdel` command deletes a user from the system. For example, entering `sudo userdel fgarcia` deletes `fgarcia` as a user. Be careful before you delete a user using this command.

The `userdel` command doesn't delete the files in the user's home directory unless you use the `-r` option. Entering `sudo userdel -r fgarcia` would delete `fgarcia` as a user and delete all files in their home directory. Before deleting any user files, you should ensure you have backups in case you need them later.

Note: Instead of deleting the user, you could consider deactivating their account with `usermod -L`. This prevents the user from logging in while still giving you access to their account and associated permissions. For example, if a user left an organization, this option would allow you to identify which files they have ownership over, so you could move this ownership to other users.

chown

The `chown` command changes ownership of a file or directory. You can use `chown` to change user or group ownership. To change the user owner of the `access.txt` file to `fgarcia`, enter `sudo chown fgarcia access.txt`. To change the group owner of `access.txt` to `security`, enter `sudo chown :security`

Module - 4

SQL & Databases

to learn
- Relational Databases
- SQL queries
- SQL filters
- SQL joints

Database: An Organized collection of information or data

Relational Database :- Contains tables that are related to each other

Primary Key : A colour where every row has unique entry

Query in SQL (Structured query Language)

query is a request for data from a database table or a combination of tables

Log : A record of event that occure within an organization's systems

Basic SQL query

There are two essential keywords in any SQL query: **SELECT** and **FROM**. You will use these keywords every time you want to query a SQL database. Using them together helps SQL identify what data you need from a database and the table you are returning it from.

The video demonstrated this SQL query:

```
SELECT employee_id, device_id

FROM employees;
```

In readings and quizzes, this course uses a sample database called the **Chinook** database to run queries. The **Chinook** database includes data that might be created at a digital media company. A security analyst employed by this company might need to query this data. For example, the database contains eleven tables, including an **employees** table, a **customers** table, and an **invoices** table. These tables include data such as names and addresses.

As an example, you can run this query to return data from the **customers** table of the **Chinook** database:

```
SELECT customerid, city, country
FROM customers;
```

Run

Reset

SELECT

The **SELECT** keyword indicates which columns to return. For example, you can return the **customerid** column from the **Chinook** database with

```
SELECT customerid
```

You can also select multiple columns by separating them with a comma. For example, if you want to return both the **customerid** and **city** columns, you should write **SELECT customerid, city**.

If you want to return all columns in a table, you can follow the **SELECT** keyword with an asterisk (*). The first line in the query will be **SELECT ***.

Note: Although the tables you're querying in this course are relatively small, using **SELECT *** may not be advisable when working with large databases and tables; in those cases, the final output may be difficult to understand and might be slow to run.

FROM

The **SELECT** keyword always comes with the **FROM** keyword. **FROM** indicates which table to query. To use the **FROM** keyword, you should write it after the **SELECT** keyword, often on a new line, and follow it with the name of the table you're querying. If you want to return all columns from the **customers** table, you can write:

```
SELECT *
```

FROM

The **SELECT** keyword always comes with the **FROM** keyword. **FROM** indicates which table to query. To use the **FROM** keyword, you should write it after the **SELECT** keyword, often on a new line, and follow it with the name of the table you're querying. If you want to return all columns from the **customers** table, you can write:

```
SELECT *
FROM customers;
```

When you want to end the query here, you put a semicolon (**;**) at the end to tell SQL that this is the entire query.

Note: Line breaks are not necessary in SQL queries, but are often used to make the query easier to understand. If you prefer, you can also write the previous query on one line as

```
SELECT * FROM customers;
```

ORDER BY

Database tables are often very complicated, and this is where other SQL keywords come in handy. **ORDER BY** is an important keyword for organizing the data you extract from a table.

ORDER BY sequences the records returned by a query based on a specified column or columns. This can be in either ascending or descending order.

Sorting in ascending order

To use the **ORDER BY** keyword, write it at the end of the query and specify a column to base the sort on. In this example, SQL will return the **customerid**, **city**, and **country** columns from the **customers** table, and the records will be sequenced by the **city** column:

```
SELECT customerid, city, country
FROM customers
ORDER BY city;
```

Run
Reset

The **ORDER BY** keyword sorts the records based on the column specified after this keyword. By default, as shown in this example, the sequence will be in ascending order. This means

- if you choose a column containing numeric data, it sorts the output from the smallest to largest. For example, if sorting on **customerid**, the ID numbers are sorted from smallest to largest.
- if the column contains alphabetic characters, such as in the example with the **city** column, it orders the records from the beginning of the alphabet to the end.

Sorting in descending order

You can also use the **ORDER BY** with the **DESC** keyword to sort in descending order. The **DESC**

default, as shown in this example, the sequence will be in ascending order. This means

- if you choose a column containing numeric data, it sorts the output from the smallest to largest. For example, if sorting on **customerid**, the ID numbers are sorted from smallest to largest.
- if the column contains alphabetic characters, such as in the example with the **city** column, it orders the records from the beginning of the alphabet to the end.

Sorting in descending order

You can also use the **ORDER BY** with the **DESC** keyword to sort in descending order. The **DESC** keyword is short for "descending" and tells SQL to sort numbers from largest to smallest, or alphabetically from Z to A. This can be done by following **ORDER BY** with the **DESC** keyword. For example, you can run this query to examine how the results differ when **DESC** is applied:

```
SELECT customerid, city, country
FROM customers
ORDER BY city DESC;
```

Run
Reset

Now, cities at the end of the alphabet are listed first.

Sorting based on multiple columns

You can also choose multiple columns to order by. For example, you might first choose the **country** and then the **city** column. SQL then sorts the output by **country**, and for rows with the same **country**, it sorts them based on **city**. You can run this to explore how SQL displays this:

```
SELECT customerid, city, country
FROM customers
ORDER BY country, city;
```

Run
Reset

Basic filters on Sql queries

operators :- A Symbol or Keyword that represents an operation

where : Indicate the condition for a filter

If it is "East%" where ___ = "East%"

It will return

everything after East

- East - 120
- East - 140
- East - 230

& ETC

Like :- used with where to search for Pattern in column.

Example :

where office like 'East%'

Example If we were to search for US but some People use USA & other use US

select *

From _____

-> where country like 'US%'

Wildcards

A **wildcard** is a special character that can be substituted with any other character. Two of the most useful wildcards are the percentage sign (%) and the underscore (_):

- The percentage sign substitutes for any number of other characters.
- The underscore symbol only substitutes for one other character.

These wildcards can be placed after a string, before a string, or in both locations depending on the pattern you're filtering for.

The following table includes these wildcards applied to the string 'a' and examples of what each pattern would return.

Pattern	Results that could be returned
'a%'	apple123, art, a
'a_'	as, an, a7
'a__'	ant, add, a1c
'%a'	pizza, Z6ra, a
'_a'	ma, 1a, Ha
'%a%'	Again, back, a

Filter Date & Number

common data types

- String :- Data consiting of orderd sequence of Character
- Numeric :- Data consisting of Numbers
- Data and times

Operators :- =, >, <, <> (Not equal), >=, <=

Between :- An operator that filters for numbers or dates within a ge

And - Both condition

Or - eighter one or Both condition

Not - Negates a Condition

table joining

employees . employee_Id

table name column name

Inner join :- Returns rows matching on a specified column that exists in more than one table

→ for Columns

Types of join

Types of Outer join

- Left join → take everything from first and return null if no match in second table
- Right join take everything from second table and return null if no match in first table
- full outer join → all return everything & "Null" where no data

→ for Rows